

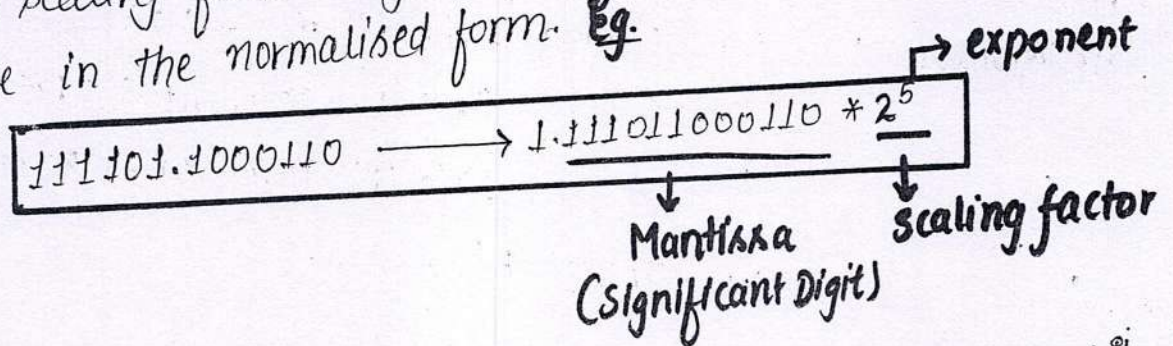
UNIT:-02

Arithmetic and Logic Unit

Floating Point Representation:- To accomodate very large integers, and very small

fractions, a computer must be able to represent numbers and operate on them in such a way that the position of the binary point is variable and is automatically adjusted as computation proceeds. In this case, the binary point is set to float, and the numbers are called floating point numbers. The floating point representation has three fields are:- sign, significant digits and exponent.

To represent the number in floating point format, first binary point is shifted to right of the first bit and the number is multiplied by the correct scaling factor to get the same value. The number is said to be in the normalised form. Eg.



The standards for representing floating point numbers in 32 bits and 64 bits have been developed by the institute of electrical and electronics engineers (IEEE).

It is also known as IEEE 754 standards.

The 32-bit standard is called a single precision representation because it occupies a single 32-bit word. The 32-bits divided into three fields:-

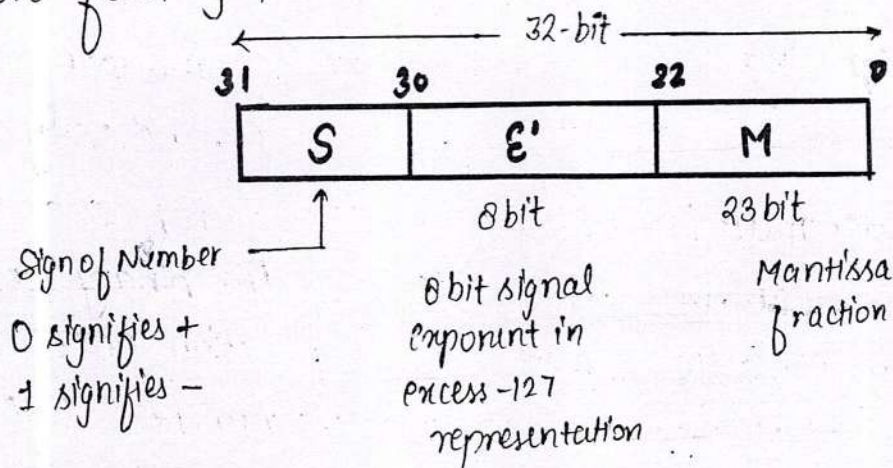
Field 1 sign, sign-bit (1-bit).

Field 2 exponent (8-bit)

Field 3 Mantissa (23-bit).

Instead of the sign exponent (E), the value actually stored in the Exponent field is $E' = E + \text{Base value}$.

In 32-bit floating point system, base is 127. Hence $E' = E(\text{scaling factor}) + 127$

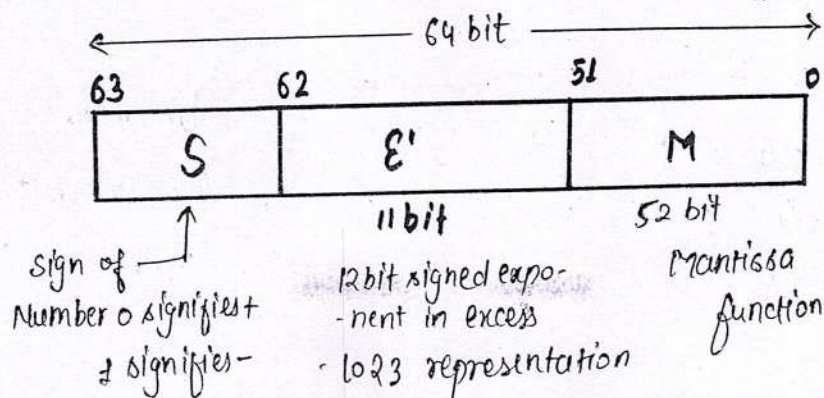


$$E' = E + 127$$

32 bit Representation

- * The 64-bit representation is called a double precision-Representation because it occupies two 32 bit words. The 64-bits are divided into three fields:
 - field (1) sign - 1 bit
 - field (2) exponent $\Rightarrow$ 11 bit
 - field (3) Mantissa $\Rightarrow$ 52 bit.

- * In double precision format, $E' = E + 1023$.



$$E' = E + 1023$$

★ Represent $(1259.125)_{10}$ in single and double precision format.

Solⁿ:

2	1259	1
2	629	1
2	314	0
2	157	1
2	78	0
2	39	1
2	19	1
2	9	1
2	4	0
2	2	0
	1	

$$.125 \times 2 = 0.250 \quad 0$$

$$0.250 \times 2 = 0.500 \quad 0$$

$$0.500 \times 2 = 1.000 \quad 1$$

$$= (10011101011.001)_2$$

$$= 1.0011101011001 \times 2^{10}$$

Single Precision Format

$$E' = E + 127$$

$$E' = 10 + 127$$

$$E' = 137$$

2	137	1
2	68	0
2	34	0
2	17	1
2	8	0
2	4	0
2	2	0
2	1	1
	0	

$\Rightarrow 10001001$

31	30	22	0
0	10001001	0011101011001	0

Double Precision Format

$$E' = E + 1023$$

$$E' = 10 + 1023$$

$$E' = 1033$$

$$E' = 1033$$

$$E' = 10000001001$$

63	62	52	0
0	10000001001	0011101011001	0

② $(-309.1875)_{10}$

$309 = (100110101)_2$

$= -(100110101.001)_2$

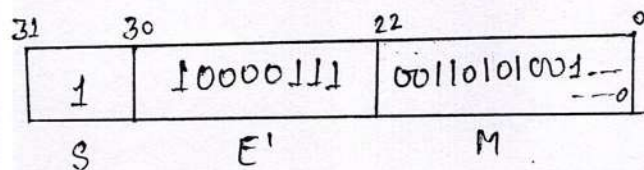
$= -1.00110101001 \times 2^8$

Single Precision

$E' = E + 127$

$E' = 8 + 127$

$E' = 135 = (10000111)_2$



$0.1875 \times 2 = 0.3750 \quad 0$

$0.3750 \times 2 = 0.7500 \quad 0$

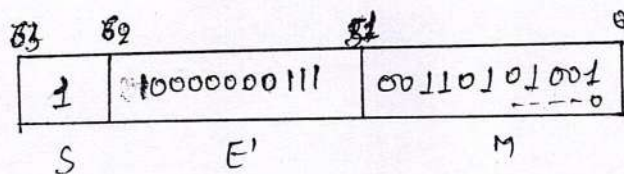
$0.7500 \times 2 = 1.5000 \quad 1$

Double Precision

$E' = E + 1023$

$E' = 8 + 1023$

$E' = 1031 = (010000000111)_2$



③ $(-79.1258)_{10}$

$79 = (1001111)_2$

$(1001111.001)_2 = 1.001111001 \times 2^6$

Single Precision

$E' = E + 127$

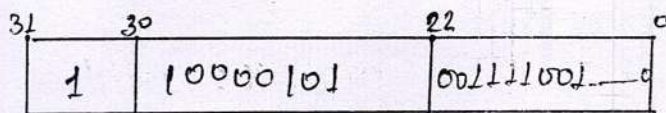
$E' = 6 + 127$

$E' = 133 = (10000101)_2$

$0.1258 \times 2 = 0.2516 \quad 0$

$0.2516 \times 2 = 0.5032 \quad 0$

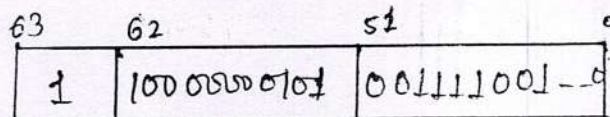
$0.5032 \times 2 = 1.0064 \quad 1$



Double Precision

$$E' = 6 + 1023$$

$$E' = 1029 = (10000000101)_2$$



4. $(1460.125)_{10}$

$$1460 = (10110110100)_2$$

$$= (10110110100.001)_2$$

$$= (1.0110110100001) \times 2^{10}$$

$$.125 \times 2 = .250 \quad 0$$

$$0.250 \times 2 = .500 \quad 0$$

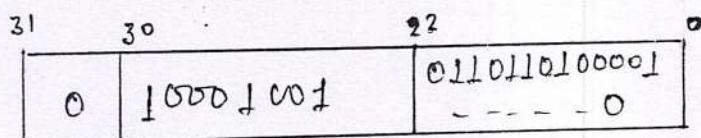
$$0.500 \times 2 = 1.00 \quad 1$$

Single Precision

$$E' = E + 127$$

$$E' = 10 + 127$$

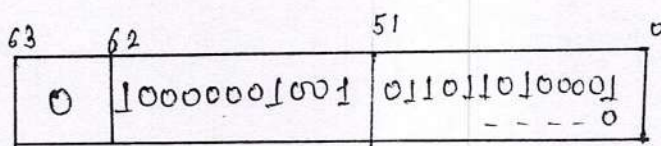
$$E' = 137 = (10001001)_2$$



Double Precision

$$E' = E + 1023 = 10 + 1023$$

$$E' = 1033 = (100000001001)_2$$



⑤ $(-1523.012)_{10}$

$(10111110011.0000)_2$

$1.0111110011 \times 2^{10}$

$E = 10$

$0.012 \times 2 = 0.024$

$0.024 \times 2 = 0.048$

$0.048 \times 2 = 0.096$

$0.096 \times 2 = 0.192$

$0.192 \times 2 = 0.384$

Single Precision

$E' = E + 127$

$E' = 10 + 127$

$E' = 137 = (10001001)_2$

1	10001001	011111001100 ----- 0
---	----------	----------------------------

Double Precision

$E' = E + 1023$

$E' = 1033 = (10000001001)_2$

1	10000001001	011111001100
---	-------------	--------------

⑥ $(1486.125)_{10}$

$= (10111001110.001)_2$

$= 1.0111001110001 \times 2^{10}$

$E = 10$

Single Precision

$E' = E + 127$

$E' = 10 + 127$

$E' = 137 = (10001001)_2$

31	30	22	0
0	10001001	0111001110001	

Double Precision

$$E' = E + 1023$$

$$E' = 10 + 1023$$

$$E' = 1033 = (100000001001)_2$$

63	62	51	0
0	100000001001	011001110001	0

$$⑦ (1243.725)_{10}$$

$$(10011011011.1011)_2$$

$$= 1.0011011011011 \times 2^{10}$$

$$E = 10$$

Single Precision

$$E' = E + 127$$

$$E' = 10 + 127$$

$$E' = 137 = (10001001)_2$$

0	10001001	0011011011011	0
---	----------	---------------	---

Double Precision

$$E' = E + 1023$$

$$E' = 10 + 1023$$

$$E' = 1033 = (100000001001)_2$$

0	100000001001	0011011011011	0
---	--------------	---------------	---

$$⑧ (-176.375)_{10}$$

$$= (10110000.011)_2$$

$$= 1.0110000011 \times 2^7$$

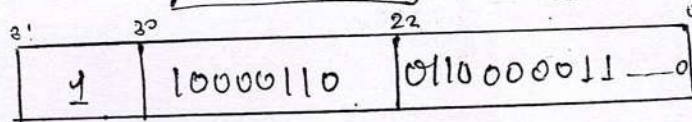
$$E = 7$$

Single Precision

$$E' = E + 127$$

$$E' = 7 + 127$$

$$E' = 134 = (10000110)_2$$

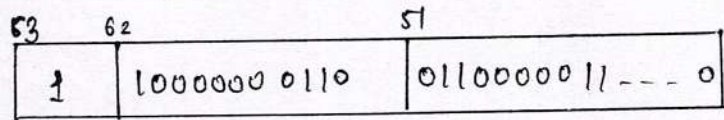


Double Precision

$$E' = E + 1023$$

$$E' = 7 + 1023$$

$$E' = 1030 = (10000000110)_2$$



② $(-1976.453)_{10}$

$$= (11110111000.0111)_2$$

$$= 1.11101110000111 \times 2^{10}$$

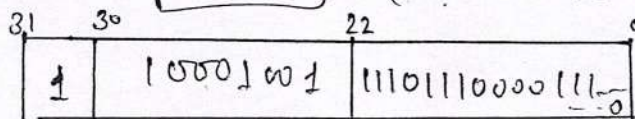
$$E = 10$$

Single Precision

$$E' = E + 127$$

$$E' = 10 + 127$$

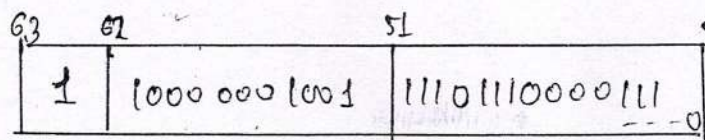
$$E' = 137 = (10001001)_2$$



Double Precision

$$E' = E + 1023 = 10 + 1023$$

$$E' = 1033 = (10000001001)_2$$



Parallel Adder

A single full-adder is capable of adding two one-bit numbers and an input carry. In order to add binary numbers with more than one bit, additional full-adders must be employed.

An n -bit, parallel adder can be constructed using number of full adder circuits connected in parallel.

Figure shows the block diagram of n -bit parallel adder using n -number of full adder circuits connected in cascade i.e., the carry output of each adder is connected to the carry input of the next higher-order adder.

It should be noted that either a half adder can be used for least significant position or the carry input of a full adder is made 0 because there is no carry into the least significant bit position.

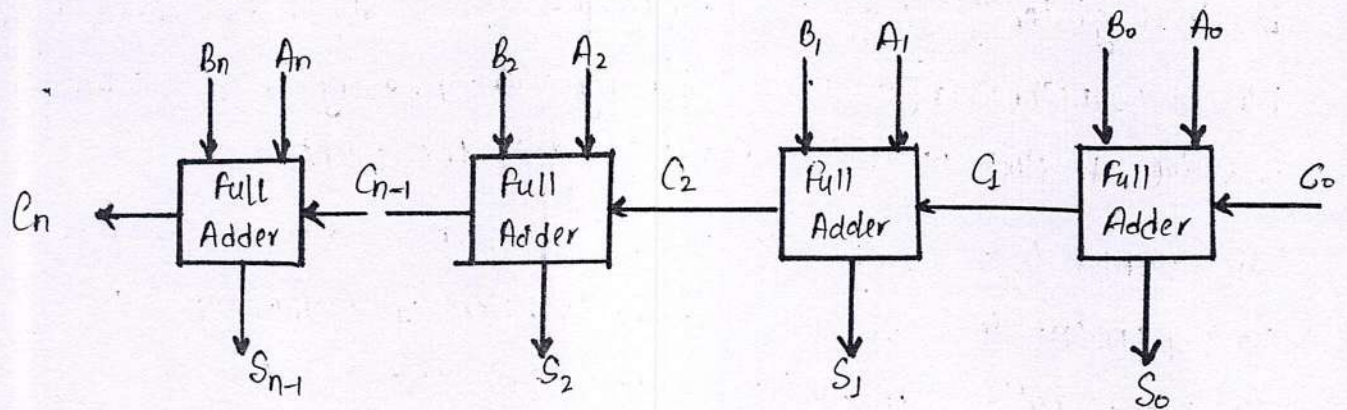


Fig. 2.6.11 - Block diagram of n -bit parallel adder

Parallel Subtractor: The subtraction of binary numbers can be done most conveniently by means of complements.

The subtraction $A - B$ can be done by taking 2's complement of B and adding it to A . The 2's complement can be obtained by taking the 1's complement and adding one to the least significant pair of bits.

The 1's complement can be implemented with inverters and a one can be added to the sum through the input carry to get 2's complement as shown in the fig.

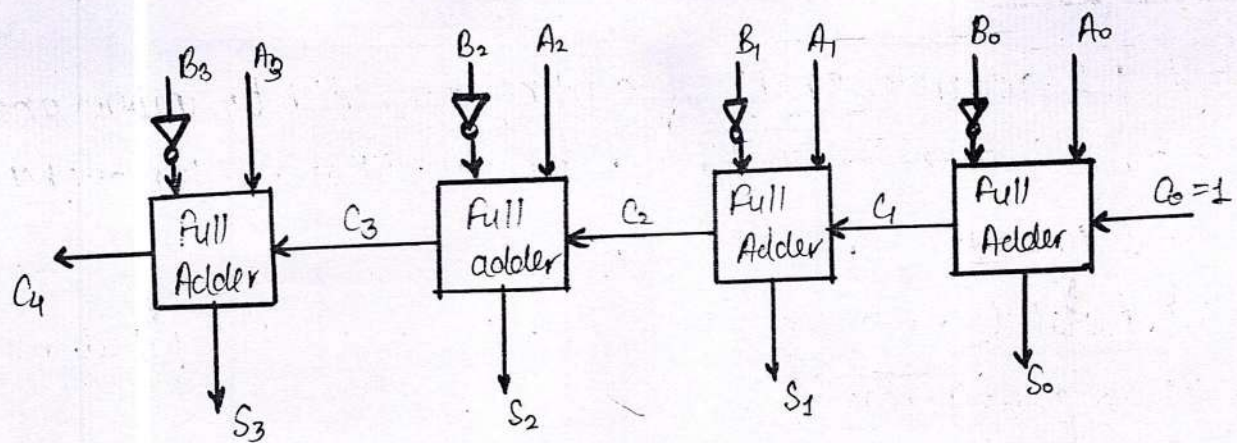


Fig. 4-bit parallel subtractor

Binary Incrementer :-

We have seen that the increment micro-operation adds one to a number in register. This micro-operation can be easily implemented with a binary counter. Every time in a binary counter, every time the count enable is active, the clock pulse transition increments the count.

The counter is a sequential circuit. The Fig. shows a combinational circuit to implement increment micro-operation. Here, one of the inputs to the least significant half-adder (HA) is connected to logic 1 and the other input is connected to the least significant bit of the number to be incremented. The output carry from lower order half adder is connected to the one of the inputs of the next higher order half adder. Such circuits get the four bit number from bits A_0 through A_3 , adds one to it, and generates the incremented output in S_0 through S_3 . If the number to be incremented is 1111, the output carry C_4 is generated and we get 0000 at S_0 through S_3 .

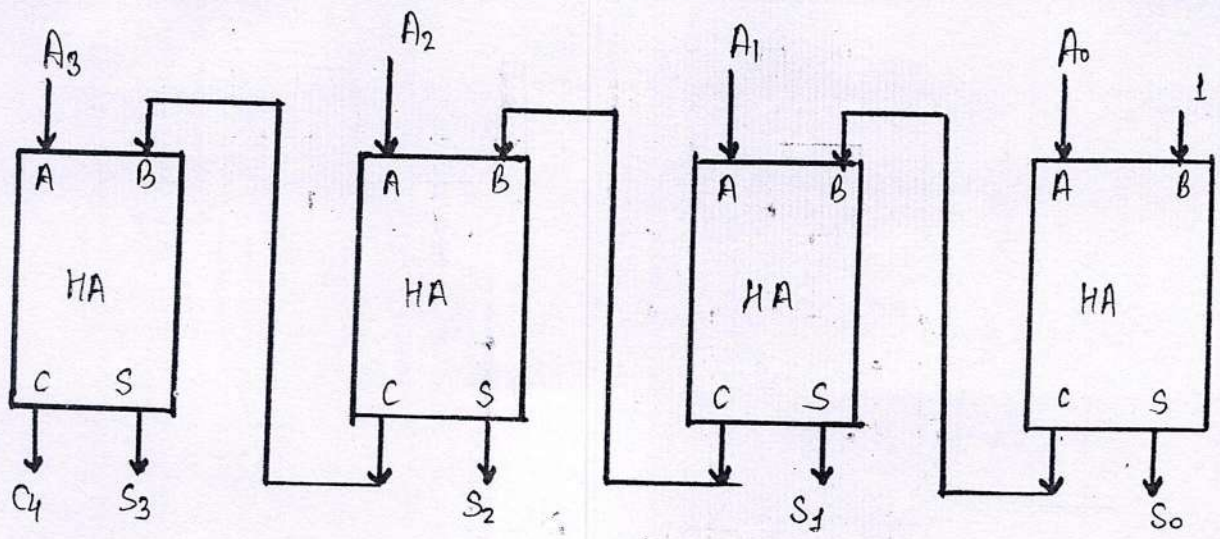


Fig. 2.6.17. 4-bit Incrementer using combinational circuit

Array Multiplier: The multiplication process of binary number is similar to decimal no.s. Actually binary multiplication is simple than decimal multiplication because it involves 0 and 1 only.

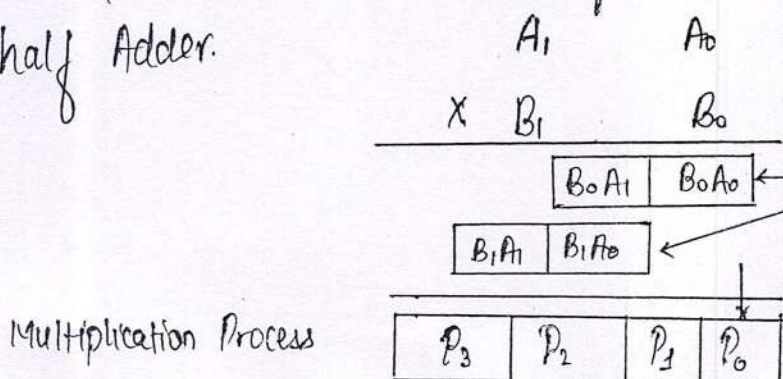
For multiplication in binary no.s, it uses n -shifts and ADDS to multiply n -bit binary number.

The combination logic circuit implemented to perform such multiplication is called combinational multiplier or array multiplier.

Let us generalize the multiplication process for 2×2 multiplier for two unsigned 2-bit no.s - multiplicand $A = A_1 A_0$ and multiplier $B = B_1 B_0$.

The multiplication process involves multiplication of 2-bit no.s and addition of 2-bit no.s.

The multiplication of two bits can be implemented using 2 input AND Gate whereas addition of 2-bits can be implemented using half Adder.



Partial products

$$P_0 = B_0 A_0$$

$$P_1 = B_0 A_1 + B_1 A_0$$

$$P_2 = B_1 A_1 + \text{Carryout of } P_1$$

$$P_3 = \text{carryout of } P_2$$

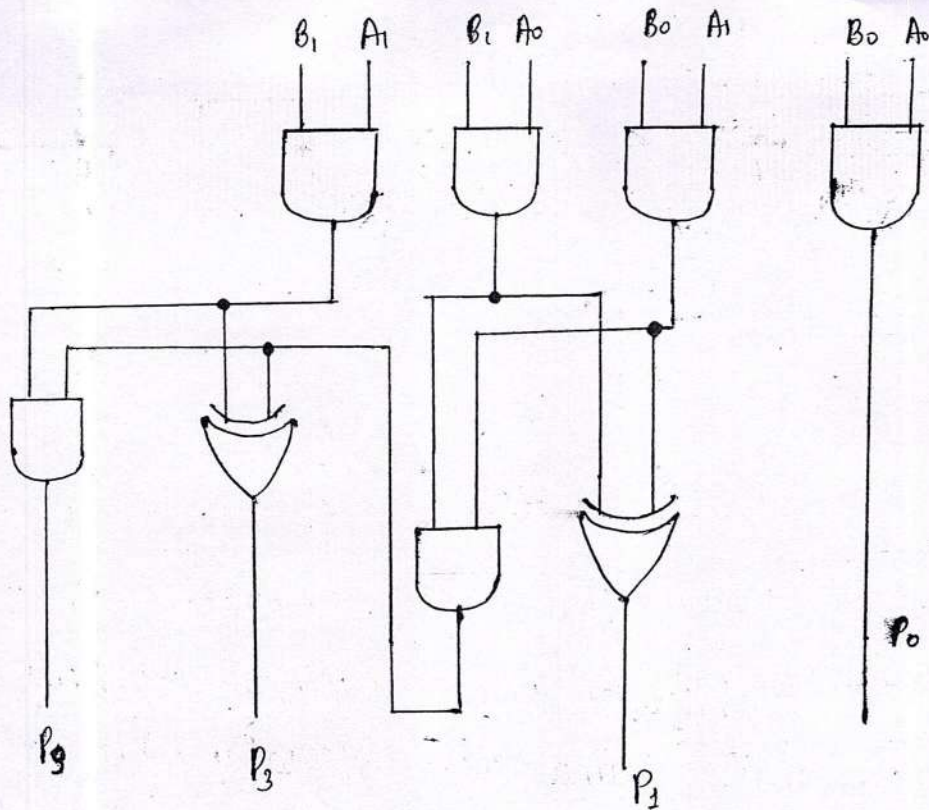
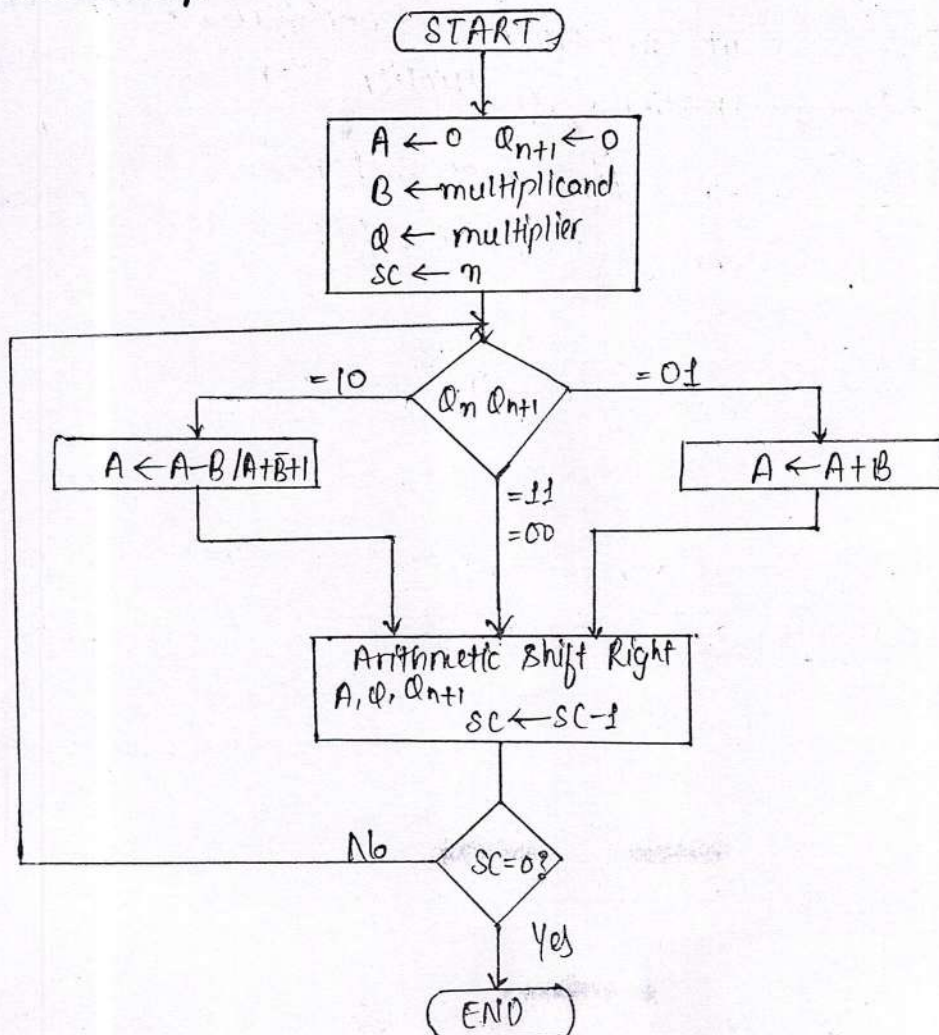


Fig 2.10.2 2x2 bit combinational array multiplier

Booth Multiplication



Algorithm:-

Step 1:- Load $A = 0$; $Q_{n+1} = 0$
 $B = \text{Multiplicand}$
 $Q = \text{multiplier}$
 $SC = n$

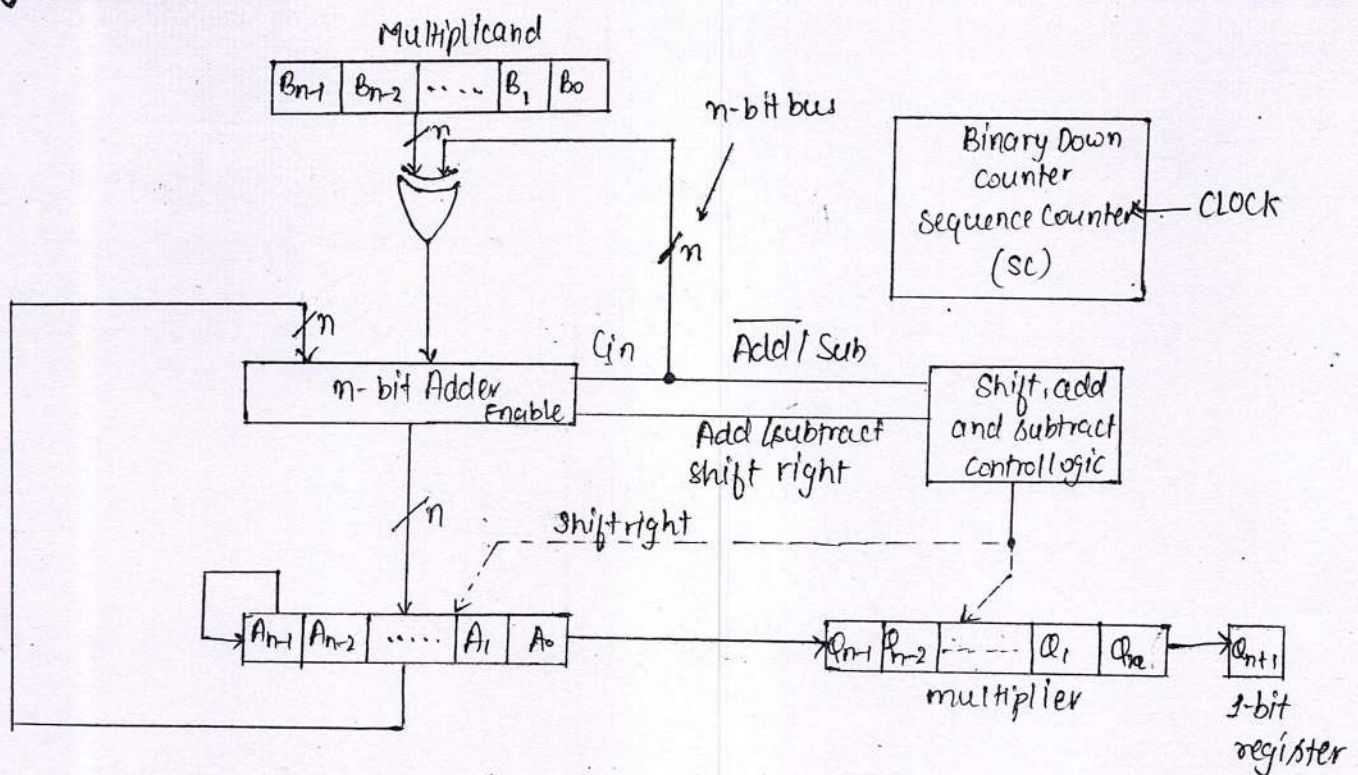
Step 2:- Check the status of $Q_n Q_{n+1}$.
 If $Q_n Q_{n+1} = 10$ perform $A \leftarrow A - B$
 If $Q_n Q_{n+1} = 01$ perform $A \leftarrow A + B$

Step 3:- Arithmetic shift right: A, Q_n, Q_{n+1}

Step 4:- Decrement sequence counter if not zero, repeat step 2 through 4.

Step 5:- Stop.

Algorithm



• Initial settings: $A \leftarrow 0$ and $Q_{n+1} = 0$

∴ Hardware implementation of signed binary multiplication

① Evaluate

(i) 9×17

$$B = 9 = 001001$$

$$\bar{B} + 1 = 110111$$

$$Q = 17 = 010001$$

Q_n	Q_{n+1}	$B = 001001$ $\bar{B} + 1 = 110111$	A	Q	Q_{n+1}	SC
		Initial	000000	010001	0	110
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 110111 \\ 110111 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	111011	101000	1	101
0	1	$A \leftarrow A + B$	$\begin{array}{r} 001001 \\ 111011 \\ \hline 000100 \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	000010	010100	0	100
0	0	$Ashr(A, Q, Q_{n+1})$	000001	001010	0	011
0	0	$Ashr(A, Q, Q_{n+1})$	000000	100101	0	010
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 110111 \\ 110111 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	111011	110010	1	001
0	1	$A \leftarrow A + B$	$\begin{array}{r} 001001 \\ 111011 \\ \hline 000100 \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	000010	011001	0	000

$$B \times Q = 000010011001$$

② $-13 * 16$

$B = -13 = 110011 \Rightarrow \bar{B}+1 = 001101$

$Q = 16 = 010000$

$+13 = 001101$

↓ 2's Complement

$= 110010 + 1$

$-13 = 110011$

Q_n	Q_{n+1}	$B = 110011$ $\bar{B}+1 = 001101$	A	Q	Q_{n+1}	SC
		Initial	000000	010000	0	110
0	0	$Ashr(A, Q, Q_{n+1})$	000000	001000	0	101
0	0	$Ashr(A, Q, Q_{n+1})$	000000	000100	0	100
0	0	$Ashr(A, Q, Q_{n+1})$	000000	000010	0	011
0	0	$Ashr(A, Q, Q_{n+1})$	000000	000001	0	010
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 001101 \\ 001101 \\ \hline 011010 \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	000110	100000	1	001
0	1	$A \leftarrow A + B$	$\begin{array}{r} 110011 \\ 111001 \\ \hline 101100 \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	111100	110000	0	000

$B * Q = 111100110000$

③ $18 * -14$

$B = 18 = 010010$

$Q = -14 = 110010$

$\bar{B} + 1 = 101101 + 1 = 101110$

$14 = 001110$

↓ 2's complement
 $= 110001 + 1$
 $= 110010$

Q_n	Q_{n+1}	$B = 010010$ $\bar{B} + 1 = 101110$	A	Q	Q_{n+1}	SC
		Initial	000000	110010	0	110
0	0	$A \leftarrow A, Q, Q_{n+1}$	000000	011001	0	101
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 101110 \\ 101110 \\ \hline \end{array}$			
		$A \leftarrow A, Q, Q_{n+1}$	110111	001100	1	100
0	1	$A \leftarrow A + B$	$\begin{array}{r} 010010 \\ 110111 \\ \hline \end{array}$			
		$A \leftarrow A, Q, Q_{n+1}$	000100	100110	0	011
0	0	$A \leftarrow A, Q, Q_{n+1}$	000010	010011	0	010
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 101110 \\ 110000 \\ \hline \end{array}$			
		$A \leftarrow A, Q, Q_{n+1}$	111000	001001	1	001
1	1	$A \leftarrow A, Q, Q_{n+1}$	111100	000100	1	000

$B * Q = 11110000100$

④ $-12 * -11$

$$B = -12 = 10100$$

$$\overline{B} + i = 0101 + 1 = 0110$$

$$Q = -11 = 10101$$

$$+12 = 01100$$

$$+11 = 01011$$

↓ 2's comp.

↓ 2's comp.

15011 + 1

10100+1

$$-12 = 10100$$

$$-11 = 10701$$

Q_n	Q_{n+1}	$B = 10100$ $B+1 = 01100$	A	Q	Q_{n+1}	SC
		Initial	00000	10101	0	101
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 01100 \\ 01100 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	00110	01010	1	100
0	1	$A \leftarrow A + B$	$\begin{array}{r} 10100 \\ 11010 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	11101	00101	0	011
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 01100 \\ 01001 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	00100	10010	1	010
0	1	$A \leftarrow A + B$	$\begin{array}{r} 10100 \\ 11000 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	11100	01001	0	001
1	0	$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 01100 \\ 01000 \\ \hline \end{array}$			
		$Ashr(A, Q, Q_{n+1})$	00100	00100	1	000
		$B * Q = 0010000100$				

Look Ahead Carry Adder (CLA) $\rightarrow$ Carry Look Ahead.

* It is also known as fast adder / high speed adder.

* In ripple carry adder, the sum and carry outputs of any stage cannot be produced until the input carry occurs, this leads to a time delay in the addition process.

* This delay is known as carry propagation delay.

Considering the example:

$$\begin{array}{r} 0101 \\ 0011 \\ + \\ \hline 1000 \end{array}$$

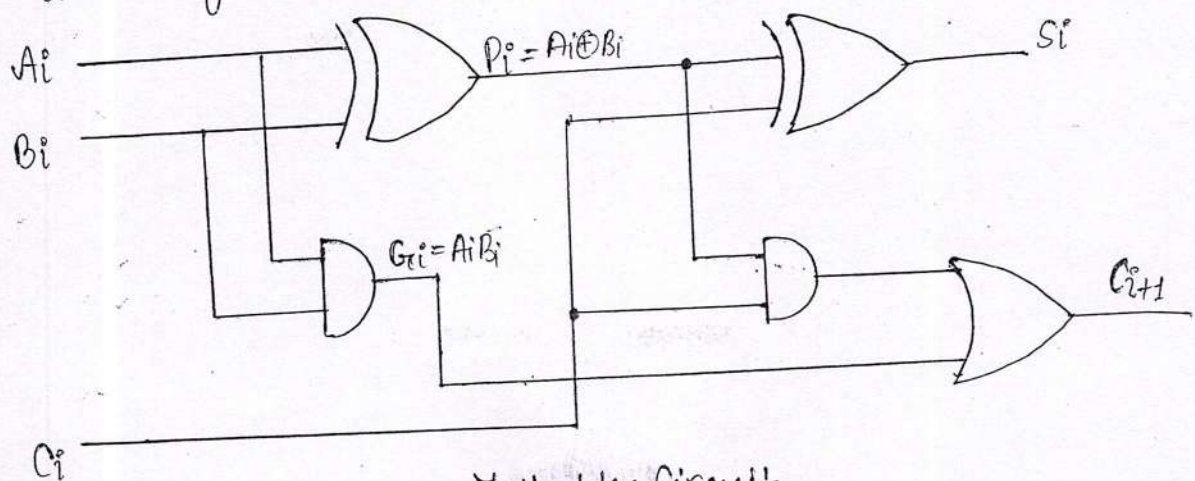
* Addition of the LSB position produces a carry into the second position.

* This carry, when added to the bits of the second position produces a carry for the third position. The latter carry when added to the bits of third position, produces a carry to the last position.

* One method of speeding up of this process by eliminating interstage carry delay is called look ahead carry addition.

* It uses two functions - carry generate (G_i) and carry propagate (P_i).

Consider the full adder which is designed with the help of two half adder



Full adder Circuit

- * We define two functions carry generate and carry propagate as follows:

$$\left. \begin{aligned} P_i &= A_i \oplus B_i \\ G_i &= A_i B_i \end{aligned} \right\} \text{--- (1)}$$

- * The output sum and carry can be expressed as:

$$\left. \begin{aligned} S_i &= P_i \oplus C_i \\ C_{i+1} &= G_i + P_i C_i \end{aligned} \right\} \text{--- (1)}$$

- * G_i is called carry generate and it produce a carry when goes both A_i and B_i are 1, regardless of the input carry.

- * P_i is called carry propagate because it is term associated with propagation of carry G_i to C_{i+1} .

- * For example, 4 stage carry look ahead adder can be defined as follows.

- * For initial condition, $C_i = C_0$. which is equal to zero.

put $i=0$ in eqⁿ (2)

$$\left. \begin{aligned} C_{i+1} &= G_i + P_i C_i \end{aligned} \right\} \text{--- (3)}$$

$$i=0; \quad C_1 = G_0 + P_0 C_0$$

where $G_0 = A_0 B_0$ and $P_0 = A_0 \oplus B_0$ and $C_0 = 0$

Hence $\rightarrow C_1 = G_0 + P_0 C_{in}$

now put $i=1$ in eqⁿ (2)

$$\begin{aligned} i=1 \quad C_2 &= G_1 + P_1 C_1 \\ C_2 &= G_1 + P_1 (G_0 + P_0 C_{in}) \end{aligned}$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \text{ --- (4)}$$

now put $i=2$ in eqⁿ (2)

$$\begin{aligned} C_3 &= G_2 + P_2 C_2 \\ C_3 &= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_{in}) \end{aligned}$$

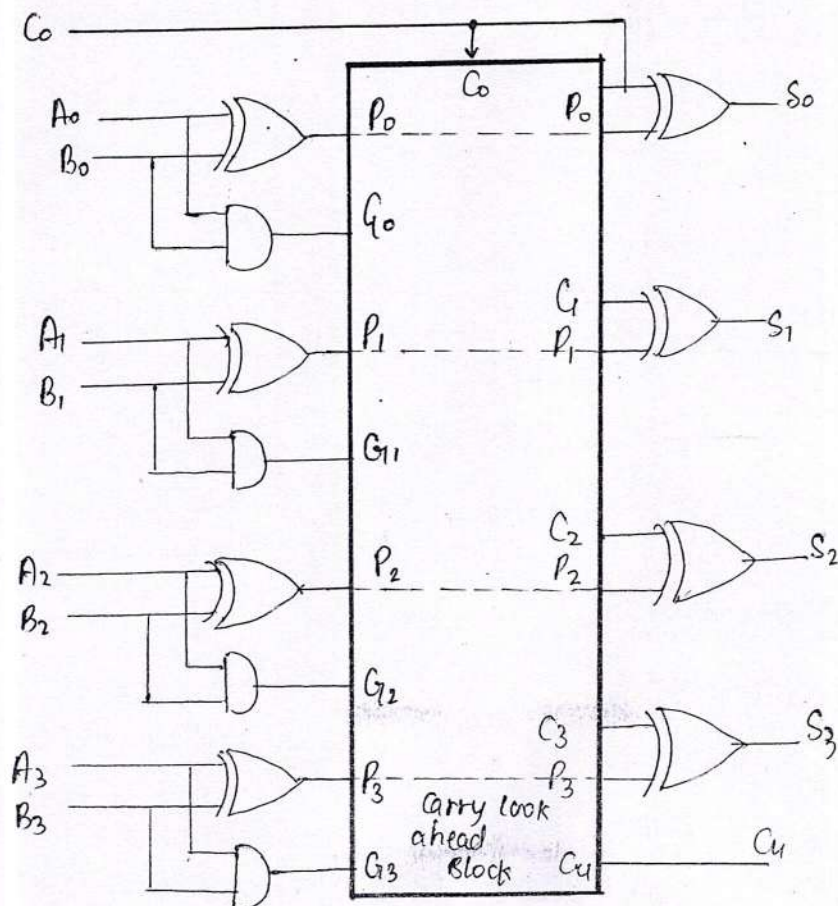
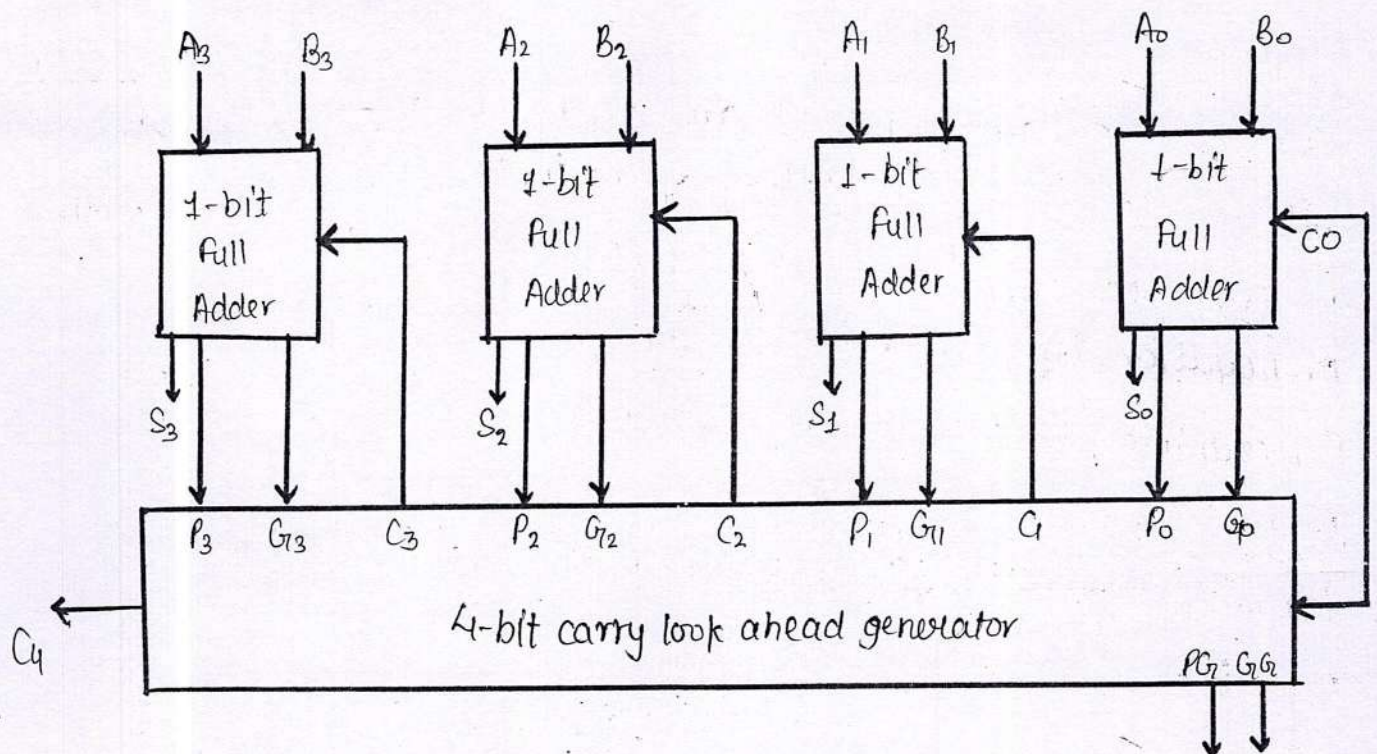
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \text{ --- (5)}$$

now put $i=3$ eqⁿ (2)

$$C_4 = G_3 + P_3 C_3$$

$$C_4 = G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in})$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \quad \text{--- (6)}$$

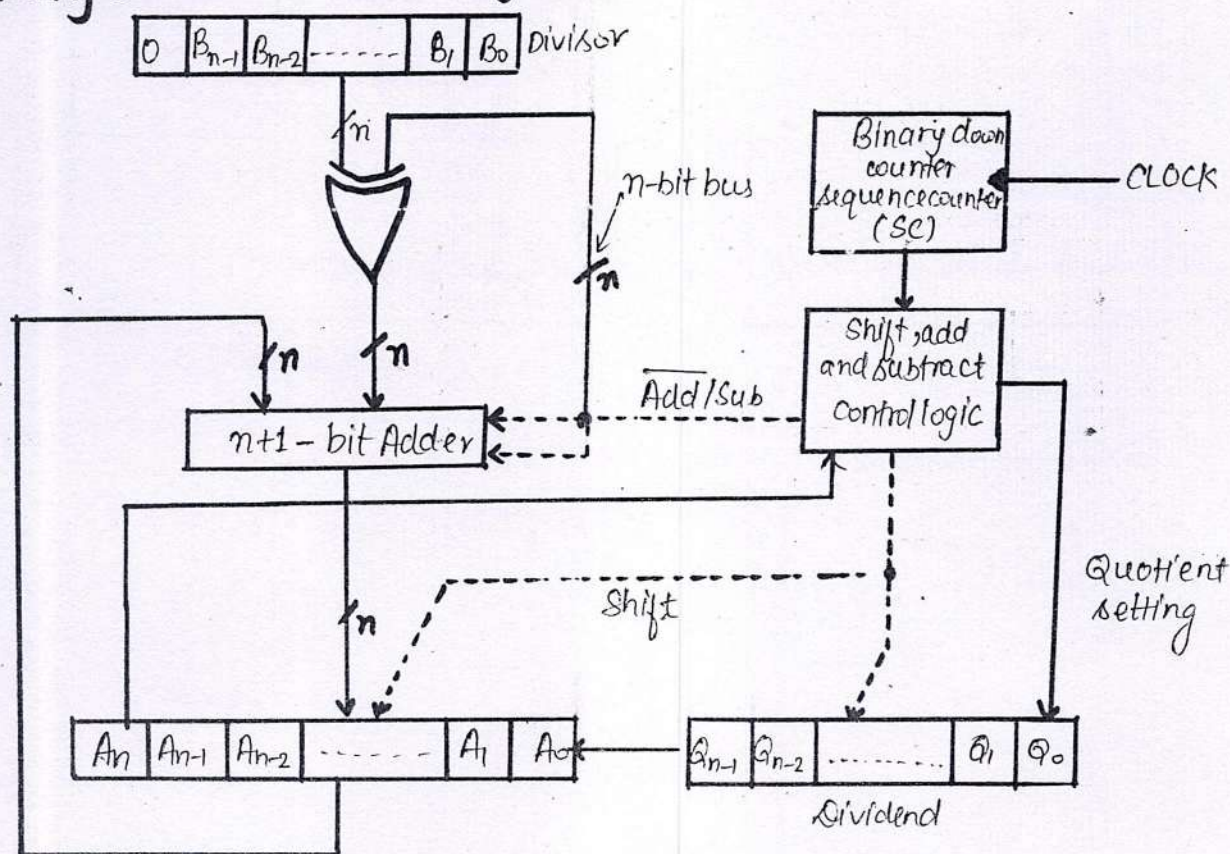


Booth's Division Algorithm

The division process for binary number is similar to decimal number. In division process, first the bits of the dividend are examined from left to right until the set of bits examined represents a no. of number greater than or equal to the divisor until this condition occurs, 0's are placed in the quotient from left to right when the condition is satisfied one is placed in the quotient and the divisor is subtracted from the partial dividend.

This result is referred as partial remainder.

Restoring Booth's Division Algorithm



Hardware to implement binary division

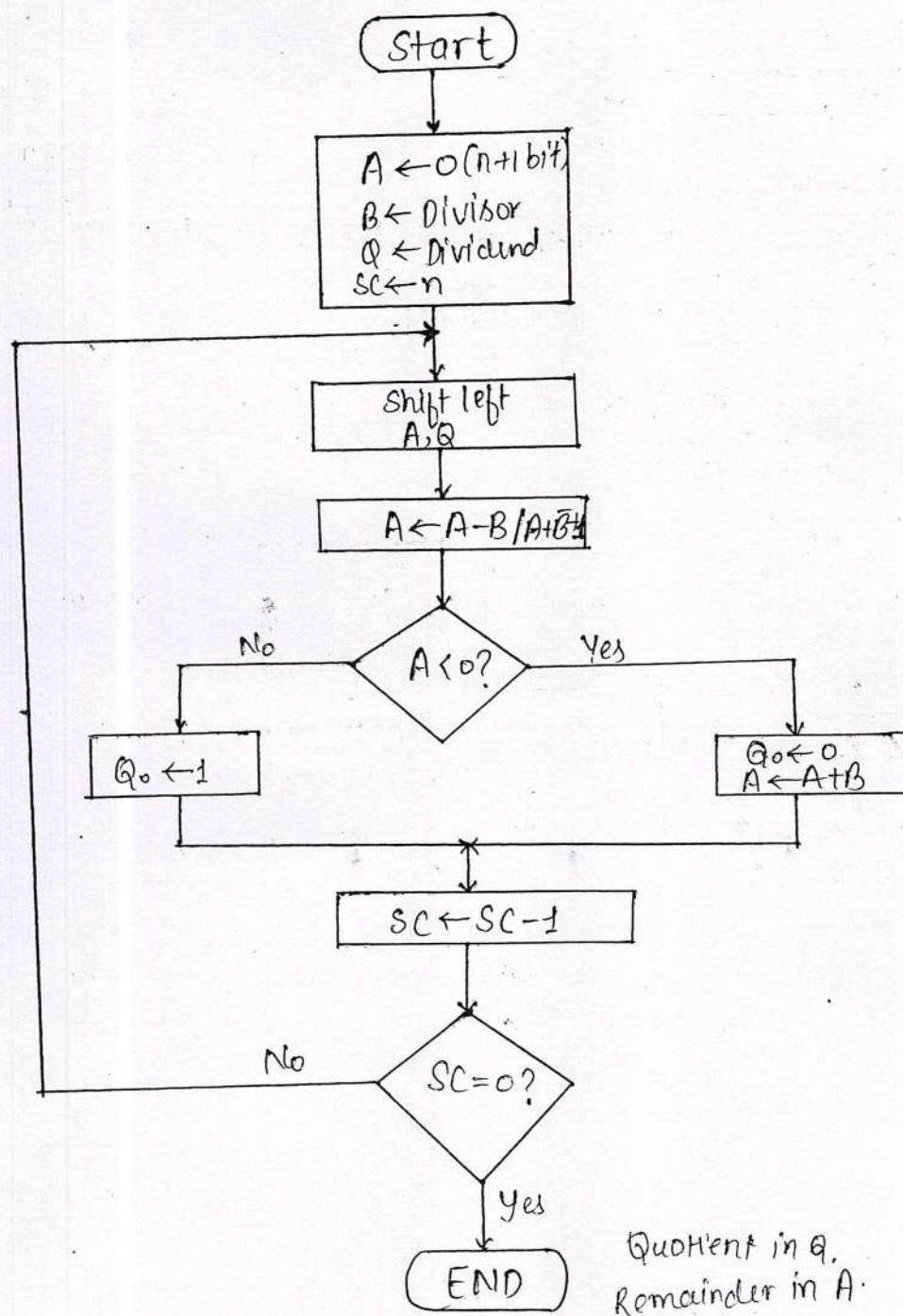
Division operation steps:

1. Shift A and Q left one binary operation position.
2. Subtract divisor (i.e. add 2's complement of divisor (B)) from A and place answer back in A ($A \leftarrow A - B$)
3. If the sign bit of A is 1, set Q_0 to 0 and add divisor back to A

(that is, restore A); otherwise, set Q_0 to 1.

4. Repeat steps 1, 2, and 3 n times.

A flowchart for division operation is as shown in fig. 2.11.3



∴ Flowchart for restoring division algorithm

Perform the division of the following nos using restoring division algorithm,

1. Dividend = 1010 and Divisor = 0011

$$\text{Divisor}(B) = 0011$$

$$\text{Dividend}(Q) = 1010$$

$$B = 00011$$

$$\bar{B} + 1 = 11101$$

Operation	A	Q	SC
Initial	00000	1010	100
Shift left(A, Q)	00001	0100	
$A \leftarrow A + \bar{B} + 1$	11101		
	<u>11110</u>		
Set $Q_0 \leftarrow 0$	00011		
$A \leftarrow A + B$	00001		011
Shift left(A, Q)	00010	1000	
	11101		
$A \leftarrow A + \bar{B} + 1$	11111		
Set $Q_0 \leftarrow 0$	00011		
$A \leftarrow A + B$	00010		010
Shift left(A, Q)	00101	0001	
$A \leftarrow A + \bar{B} + 1$	11101		
Set $Q_0 \leftarrow 1$	00000		001
Shift left(A, Q)	00100	0011	
$A \leftarrow A + \bar{B} + 1$	11101		
	<u>00001</u>		000

Ans

$$\text{Quotient} = 0011$$

$$\text{Remainder} = 00001$$

(ii) $17 \div 3$

$$B = 3 = 000011$$

$$Q = 17 = 10001$$

$$B = 000011$$

$$\bar{B} + 1 = 111101$$

Operation	A	Q	SC
Initial	000000	10001	101
Shift Left (A, Q)	000001	100010	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000001 \\ 111101 \\ \hline 111110 \end{array}$		
Set $Q_0 \leftarrow 0$	$\begin{array}{r} 000011 \\ \hline 000001 \end{array}$		100
$A \leftarrow A + B$			
Shift left (A, Q)	000010	00100	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000010 \\ 111101 \\ \hline 111111 \end{array}$		
Set $Q_0 \leftarrow 0$	$\begin{array}{r} 000011 \\ \hline 000010 \end{array}$		01100
$A \leftarrow A + B$			
Shift Left (A, Q)	000100	01001	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000100 \\ 111101 \\ \hline 000001 \end{array}$		010
Set $Q_0 \leftarrow 1$			
Shift Left (A, Q)	000010	10010	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000010 \\ 111101 \\ \hline 011111 \end{array}$		
Set $Q_0 \leftarrow 0$	$\begin{array}{r} 011111 \\ \hline 000011 \end{array}$		
$A \leftarrow A + B$	$\begin{array}{r} 000011 \\ \hline 000010 \end{array}$		

001

Shift left(A, Q)

000101

00101

$A + \bar{B} + 1$

$$\begin{array}{r} 111101 \\ \hline 000010 \end{array}$$

000

Set $Q_0 \leftarrow 1$

$17/3 = 000010 \Rightarrow \text{Quotient}$

$00101 \Rightarrow \text{Remainder}$

(ii) $1101/101$

$Q = 1101$

$B = 00101$

$\bar{B} + 1 = 11011$

Operation.

A

Q

SC

initial

00000

1101

100

Shift left(A, Q)

00001

1010

$A \leftarrow A + \bar{B} + 1$

$$\begin{array}{r} 11011 \\ \hline 01100 \end{array}$$

Set $Q_0 \leftarrow 0$

$A \leftarrow A + B$

$$\begin{array}{r} 00101 \\ \hline 00001 \end{array}$$

011

Shift left(A, Q)

00011

0100

$A \leftarrow A + \bar{B} + 1$

$$\begin{array}{r} 11011 \\ \hline 01110 \end{array}$$

Set $Q_0 \leftarrow 0$

$A \leftarrow A + B$

$$\begin{array}{r} 00101 \\ \hline 00011 \end{array}$$

010

Shift left(A, Q)

00110

1001

$A \leftarrow A + \bar{B} + 1$

$$\begin{array}{r} 11011 \\ \hline 00001 \end{array}$$

Set $Q_0 \leftarrow 1$

001

Shift Left (A, Q)

$$A \leftarrow A + \bar{B} + 1$$

$$\text{Set } Q_0 \leftarrow 0$$

$$A \leftarrow A + B$$

00011

11011

11110

00101

00011

0010

000

Remainder $\Rightarrow$ 00011

Quotient = 0010

(iv) 1000/111

$$Q = 1000$$

$$B = 0011$$

$$\bar{B} + 1 = 11001$$

Operation	A	Q	SC
-initial	00000	1000	100
Shift Left (A, Q)	00001	0000	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 11001 \\ 01010 \\ \hline \end{array}$		
Set $Q_0 \leftarrow 0$			
$A \leftarrow A + B$	$\begin{array}{r} 00111 \\ 00001 \\ \hline \end{array}$		011
Shift Left (A, Q)	00010	0000	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 11001 \\ 01011 \\ \hline \end{array}$		
Set $Q_0 \leftarrow 0$			
$A \leftarrow A + B$	$\begin{array}{r} 00111 \\ 00010 \\ \hline \end{array}$		010
Shift Left (A, Q)	00100	0000	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 11001 \\ 01101 \\ \hline \end{array}$		010
Set $Q_0 \leftarrow 0$			

$$A \leftarrow A+B$$

$$\begin{array}{r} 11101 \\ 00111 \\ \hline 00100 \end{array}$$

Shift Left (A, Q)

$$01000$$

$$0001$$

$$A \leftarrow A+\bar{B}+1$$

$$\begin{array}{r} 11001 \\ \hline 00001 \end{array}$$

Set $Q_0 \leftarrow 1$

$$A \leftarrow A+B$$

$$000$$

Shift Left (A, Q)

$$00010$$

$$0010$$

$$A \leftarrow A+\bar{B}+1$$

$$\begin{array}{r} 11001 \\ \hline 01011 \end{array}$$

Set $Q_0 \leftarrow 0$

$$00111$$

$$A \leftarrow A+B$$

$$00010$$

$$000$$

~~Set~~

$$\text{Quotient} = 00010$$

$$\text{Remainder} = 00001$$

Non-Restoring Division

Consider the sequence of operations that takes place after the subtraction operation in the restoring algorithm.

If A is positive

Shift Left and subtract
divisor $\rightarrow 2A-B$

If A is negative

Restore $\rightarrow A+B$

Shift Left and subtract
divisor $\rightarrow 2(A+B)-B$
 $= 2A+B$

Looking at the above operations that takes place after the subtraction operation is

Looking at the above operations, we can write following steps for non-restoring algorithm.

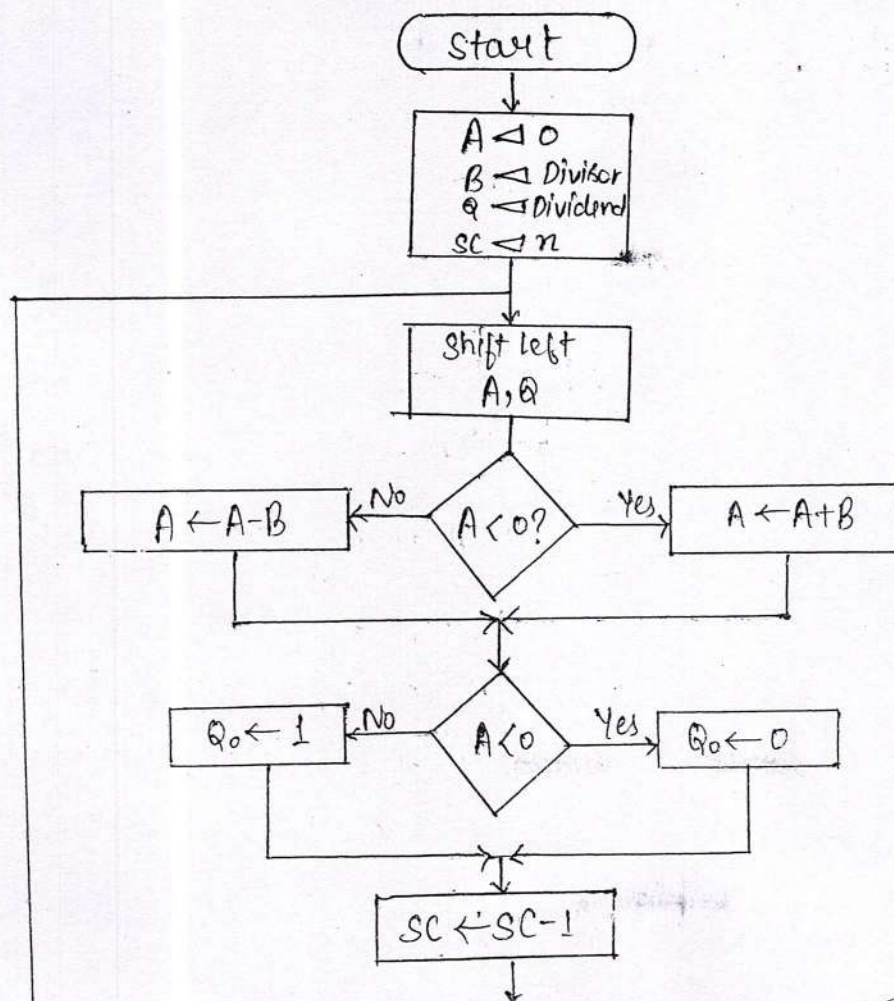
Step 1: If the sign of A is 0, shift A and Q left one bit position and subtract divisor from A, otherwise, shift A and Q left and add divisor to A. If the sign of A is 0, set Q_0 to 1, otherwise, set Q_0 to 0.

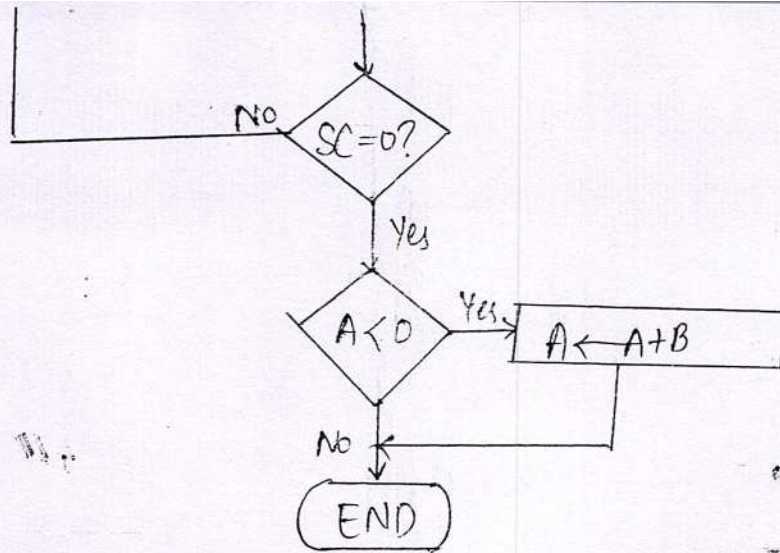
Step 2: Repeat steps 1 and 2 for n times.

Step 3: If the sign of $A \neq 1$, add divisor to A.

Note: Step 3 is required to leave the proper positive remainder in A at the end of n cycles.

A flowchart for non-restoring division operation is as shown in Fig 2.11.10





Quotient in Q
Remainder in A.

(i) 1101 / 11

$Q = 1101$

$B = 0011$

$B = 00011$

$B+1 = 11101$

Operation	A	Q	SC
Initial	00000	1101	100
Shift left A, Q	00001	1010	
$A \leftarrow A+B+1$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 11101 \\ \underline{11110} \end{array}$		011
Shift Left (A, Q)	$\begin{array}{r} 11101 \\ \underline{00011} \\ 00000 \end{array}$	0101	
$A+B$ Set $Q_0 \leftarrow 1$	$\begin{array}{r} 11101 \\ \underline{00011} \\ 00000 \end{array}$		
Shift left (A, Q)	$\begin{array}{r} 00000 \\ \underline{11101} \\ 11101 \end{array}$	1010	
$A \leftarrow A+B+1$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 00000 \\ \underline{11101} \\ 11101 \end{array}$		001
Shift Left (A, Q)	$\begin{array}{r} 11011 \\ \underline{00011} \\ 11110 \end{array}$	0100	
$A \leftarrow A+B$	$\begin{array}{r} 11011 \\ \underline{00011} \\ 11110 \end{array}$		000
$A+B$	$\begin{array}{r} 00011 \\ \underline{00001} \end{array}$		

$Q = 100$
 $R = 00001$

(ii) 17/3

$$Q = 17 = 10001$$

$$B = 3 = 00011$$

$$B = 000011$$

$$\bar{B} + 1 = 111101$$

Operation	A	Q	SC
initial	000000	10001	101
Shift Left (A, Q)	000001	00010	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000001 \\ 111101 \\ \hline 111110 \end{array}$		
Set $Q_0 \leftarrow 0$			100
Shift Left (A, Q)	111100	00100	
$A \leftarrow A + B$	$\begin{array}{r} 111100 \\ 000011 \\ \hline 111111 \end{array}$		011
Set $Q_0 \leftarrow 0$			
Shift Left (A, Q)	111110	01001	
$A \leftarrow A + B$	$\begin{array}{r} 111110 \\ 000011 \\ \hline 000001 \end{array}$		
Set $Q_0 \leftarrow 1$			010
Shift Left (A, Q)	000010	10010	
$A \leftarrow A + \bar{B} + 1$	$\begin{array}{r} 000010 \\ 111101 \\ \hline 111111 \end{array}$		
Set $Q_0 \leftarrow 0$			001
Shift Left (A, Q)	111111	00101	
$A \leftarrow A + B$	$\begin{array}{r} 111111 \\ 000011 \\ \hline 000010 \end{array}$		
Set $Q_0 \leftarrow 1$			000

(iii) 19/4

$$Q = 19 = 10011$$

$$B = 00100$$

$$B = 000100$$

$$\bar{B} + 1 = 111011 + 1$$

$$\bar{B} + 1 = 111100$$

Operation	A	Q	SC
Initial	000000	10011	101
Shift Left (A, Q) $A \leftarrow A + \bar{B} + 1$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 000000 \\ 000001 \\ 111100 \\ \hline 111101 \end{array}$	00110	100
Shift Left (A, Q) $A \leftarrow A + B$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 111010 \\ 000100 \\ \hline 111110 \end{array}$	01100	011
Shift Left (A, Q) $A \leftarrow A + B$ Set $Q_0 \leftarrow 1$	$\begin{array}{r} 111100 \\ 000100 \\ \hline 000000 \end{array}$	11001	010
Shift Left (A, Q) $A \leftarrow A + \bar{B} + 1$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 000001 \\ 111100 \\ \hline 111101 \end{array}$	10010	001
Shift Left (A, Q) $A \leftarrow A + B$ Set $Q_0 \leftarrow 0$	$\begin{array}{r} 111011 \\ 000100 \\ \hline 111111 \end{array}$	100100	000

A+B

$$\begin{array}{r} 000100 \\ 000011 \\ \hline \end{array}$$

Comparison b/w restoring and non-restoring algorithm

Restoring

Needs restoring of register A if the result of subtraction is negative.

In each cycle content of register A is first shifted left and then divisor is subtracted from it.

Does not need restoring of remainder.

Slower algorithm

Non-restoring

Does not need restoring

In each cycle content of register A is first shifted left and then division is added or subtracted with the contents of register A depending on the sign of A.

Needs restoring of remainder if remainder is negative

Faster algorithm